Prüfungsrelevante Lernkarten

Software Engineering · Spaced Repetition

Kompakte Frage-Antwort-Karten aus den Unterlagen in FH/klausur. Die vom Lernauftrag ausgeschlossenen Themen sind bewusst nicht enthalten.

Lernhinweis

Beantworte jede Frage zuerst aus dem Gedächtnis. Decke die Antwort ab, formuliere sie in eigenen Worten und wiederhole schwierige Karten in kürzeren Abständen.

Softwarequalität, Fehlerbegriffe und Testebenen

01 · Frage
Was beweist formale Verifikation?

Antwort

Sie beweist mathematisch, dass ein Programm eine formale Spezifikation erfüllt. Sie ist aufwendig und wird vor allem bei besonders kritischen Systemen eingesetzt.

02 · Frage
Was kann dynamisches Testen zeigen und was kann es nicht beweisen?

Antwort

Es kann zeigen, dass konkrete Eingaben funktionieren oder dass ein Fehler auftritt. Vollständige Fehlerfreiheit kann es nicht beweisen.

03 · Frage
Wie unterscheiden sich Error, Fault, Failure und Incident?

Antwort

Error ist eine menschliche Handlung mit falschem Ergebnis. Fault ist die daraus hervorgehende fehlerhafte Stelle im System. Failure ist die Nichterfüllung einer Anforderung durch das System. Incident ist das vom Benutzer wahrgenommene Symptom.

04 · Frage
Wie lautet die typische Fehlerkette?

Antwort

Error → Fault → Failure → Incident.

05 · Frage
Was testet ein Unit-Test?

Antwort

Eine kleine, klar abgegrenzte Einheit, häufig eine einzelne Methode oder Klasse. Unit-Tests sind meist schnell, einfach zu automatisieren und überwiegend White-Box-Tests.

06 · Frage
Was testet ein Integrationstest?

Antwort

Das Zusammenspiel mehrerer Komponenten, Klassen, Schichten oder externer Systeme.

07 · Frage
Was testet ein Systemtest?

Antwort

Das vollständige System gegen seine Anforderungen und Spezifikation. Systemtests sind oft umfassend, komplex, langsam und überwiegend Black-Box-Tests.

08 · Frage
Wie ist die Testpyramide aufgebaut?

Antwort

Aus vielen schnellen und günstigen Unit-Tests, weniger Integrationstests und wenigen langsamen, komplexen und teuren Systemtests.

09 · Frage
Warum sollen Fehler möglichst früh gefunden werden?

Antwort
Der Kontext der Änderung ist noch präsent, die Ursache lässt sich leichter identifizieren und die Behebung ist schneller und günstiger.

10 · Frage
Wer trägt Verantwortung für die Produktqualität?

Antwort
Das gesamte Entwicklungsteam beziehungsweise insbesondere die Entwickler, nicht ausschließlich eine separate Testabteilung.

Unit Testing und testbares Design

11 · Frage
Was bedeutet SUT?

Antwort
System Under Test: die Klasse, Methode oder Einheit, die durch einen Test geprüft wird.

12 · Frage
Welche Vorteile bieten Unit-Tests?

Antwort
Schnelles Feedback, Schutz vor Regressionen, Sicherheit beim Refactoring, ausführbare Dokumentation, Unterstützung testbaren Designs und Fokus auf das fachliche Was statt auf das technische Wie.

13 · Frage
Was ist Testbarkeit?

Antwort
Die Eigenschaft eines Systems, leicht isoliert, kontrolliert und zuverlässig getestet werden zu können. Sie ist ein Qualitätsmerkmal der Wartbarkeit.

14 · Frage
Welche Faktoren verschlechtern die Testbarkeit?

Antwort
Nicht isolierbare Einheiten, starke Kopplung, unflexible Initialisierung, verborgene Zustände, nicht erreichbare Kollaboratoren, fehlende Kontrolle und nicht beobachtbare Ergebnisse.

15 · Frage
Was bedeutet SRP?

Antwort
Single Responsibility Principle: Eine Softwareeinheit soll eine einzelne, klar abgegrenzte Verantwortung beziehungsweise nur einen wesentlichen Grund für Änderungen haben.

16 · Frage
Was bedeutet OCP?

Antwort
Open/Closed Principle: Softwareeinheiten sollen für Erweiterungen offen, für Änderungen am bestehenden Verhalten aber möglichst geschlossen sein.

17 · Frage
Was bedeutet LSP?

Antwort

Liskov Substitution Principle: Untertypen müssen ihre Basistypen ersetzen können, ohne grundlegende Erwartungen an deren Verhalten zu verletzen.

18 · Frage
Was bedeutet ISP?

Antwort

Interface Segregation Principle: Clients sollen nicht von Methoden abhängen müssen, die sie nicht verwenden. Kleine, spezifische Schnittstellen reduzieren Kopplung.

19 · Frage
Was bedeutet DIP?

Antwort

Dependency Inversion Principle: Übergeordnete und untergeordnete Module sollen von Abstraktionen abhängen, nicht voneinander. Abhängigkeiten werden beispielsweise per Dependency Injection gesetzt.

20 · Frage
Was bedeutet FIRST bei Unit-Tests?

Antwort

Fast, Independent, Repeatable, Self-validating, Timely: schnell, unabhängig, wiederholbar, selbstvalidierend und rechtzeitig geschrieben.

21 · Frage
Was bedeutet AAA?

Antwort

Arrange, Act, Assert: Test und Abhängigkeiten vorbereiten, die zu testende Funktion ausführen und das erwartete Ergebnis oder die relevante Interaktion prüfen.

22 · Frage
Was passiert im Arrange-Teil?

Antwort

Das SUT, Eingaben, Anfangszustände und benötigte Test-Doubles werden vorbereitet.

23 · Frage
Was passiert im Act-Teil?

Antwort

Die zu testende Funktion wird ausgeführt, möglichst genau einmal.

24 · Frage
Was passiert im Assert-Teil?

Antwort

Erwartete Werte, Zustände, Exceptions oder fachlich relevante Interaktionen werden überprüft.

25 · Frage
Was ist Äquivalenzklassenbildung?

Antwort
Eingaben werden in Klassen eingeteilt, die voraussichtlich gleich verarbeitet werden. Aus jeder gültigen und ungültigen Klasse wird mindestens ein repräsentativer Wert getestet.

26 · Frage
Was ist Grenzwertanalyse?

Antwort
Das gezielte Testen von Werten an und unmittelbar außerhalb der Grenzen von Eingabeklassen.

27 · Frage
Welche Werte sind bei einem erlaubten Bereich von 18 bis 70 besonders wichtig?

Antwort
17, 18, 70 und 71: die beiden Grenzwerte und die jeweils angrenzenden ungültigen Werte.

28 · Frage
Was ist ein Exploration Test?

Antwort
Ein Test, mit dem das tatsächliche Verhalten einer unbekannten Bibliothek oder API erkundet und verstanden wird.

Test-Doubles und Moq

29 · Frage
Warum verwendet man Test-Doubles?

Antwort
Um Kollaboratoren des SUT zu ersetzen und eine Einheit isoliert, schnell und kontrolliert testen zu können.

30 · Frage
Was ist ein Dummy?

Antwort
Ein Objekt, das nur übergeben wird, damit eine Schnittstelle erfüllt ist. Sein Inhalt und Verhalten sind für den Test nicht relevant.

31 · Frage
Was ist ein Stub?

Antwort
Ein Test-Double, das für bestimmte Aufrufe festgelegte Rückgabewerte liefert.

32 · Frage
Was ist ein Spy?

Antwort
Ein Test-Double, das zusätzlich Interaktionen aufzeichnet. Es ist ein Stub mit Beobachtungsfunktion.

33 · Frage
Was ist ein Fake?

Antwort
Eine vereinfachte, aber funktionsfähige Implementierung, zum Beispiel eine Datenbank im Arbeitsspeicher.

34 · Frage
Was ist ein Mock?

Antwort

Ein Ersatzobjekt, mit dem erwartete Interaktionen definiert und verifiziert werden. Mocks werden häufig durch ein Mocking-Framework erzeugt.

35 · Frage
Was ist State Verification?

Antwort

Die Prüfung, ob das SUT nach der Ausführung den erwarteten fachlichen Zustand erreicht hat. Dazu werden Ist- und Sollwerte verglichen.

36 · Frage
Was ist Behavior Verification?

Antwort

Die Prüfung, ob das SUT bestimmte Interaktionen mit einem Kollaborator ausgeführt hat. Dafür werden Mocks oder Spies verwendet.

37 · Frage
Warum sind Behavior-Tests oft fragiler als State-Tests?

Antwort

Sie prüfen konkrete interne Abläufe und koppeln den Test dadurch stärker an die Implementierung statt an das fachliche Ergebnis.

38 · Frage
Wie lautet der typische Moq-Ablauf?

Antwort

Mock erzeugen, falls nötig Verhalten mit `Setup` festlegen, SUT ausführen und relevante Interaktionen mit `Verify` prüfen.

39 · Frage
Wofür dient `Setup` in Moq?

Antwort

Es legt fest, wie ein Mock auf einen bestimmten Aufruf reagieren soll, zum Beispiel mit einem Rückgabewert.

40 · Frage
Was bedeutet `It.IsAny<T>()`?

Antwort

Der Argument-Matcher akzeptiert jeden Wert des Typs `T`.

41 · Frage
Wofür dient `It.Is<T>(...)`?

Antwort

Es akzeptiert nur Argumente, die eine selbst definierte Bedingung erfüllen.

42 · Frage
Wofür dient `SetupSequence`?

Antwort

Damit werden mehrere aufeinanderfolgende Rückgabewerte für wiederholte Aufrufe festgelegt.

43 · Frage
Wozu dient `Throws` in Moq?

Antwort

Damit wird simuliert, dass eine Abhängigkeit bei einem bestimmten Aufruf eine Exception auslöst.

44 · Frage
Was macht `Verify`?

Antwort

Es prüft, ob eine erwartete Interaktion mit dem Mock stattgefunden hat, gegebenenfalls mit einer bestimmten Aufrufanzahl.

45 · Frage
Was ist der Unterschied zwischen Loose- und Strict-Mock?

Antwort

Ein Loose-Mock liefert für nicht vorbereitete Aufrufe Standardwerte. Ein Strict-Mock wirft bei nicht vorbereiteten Aufrufen eine Exception.

46 · Frage
Was bedeutet Oververification?

Antwort

Es werden zu viele oder zu genaue Implementierungsdetails verifiziert. Dadurch werden Tests unnötig fragil.

TDD und Testqualität

47 · Frage
Wie lautet der TDD-Zyklus?

Antwort

Red – Green – Refactor: einen fehlschlagenden Test schreiben, minimalen Produktionscode ergänzen und danach den Code verbessern.

48 · Frage
Was besagen die drei TDD-Gesetze?

Antwort

Vor Produktionscode existiert ein fehlschlagender Test; es wird nur so viel Testcode geschrieben, dass er fehlschlägt; danach wird nur so viel Produktionscode ergänzt, dass der aktuelle Test besteht.

49 · Frage
Was ist ein Obscure Test?

Antwort

Ein Test, dessen Zweck oder Ablauf nicht sofort verständlich ist.

50 · Frage
Was ist Assertion Roulette?

Antwort

Ein Test enthält mehrere Assertions, aber es ist unklar, welche Assertion den Fehlschlag verursacht hat.

51 · Frage
Was ist ein fragiler Test?

Antwort
Ein Test, der bereits bei einer fachlich harmlosen Änderung oder einem Refactoring fehlschlägt.

52 · Frage
Was ist ein erratischer Test?

Antwort
Ein Test, der scheinbar zufällig besteht oder fehlschlägt.

53 · Frage
Warum ist Testcode genauso zu pflegen wie Produktionscode?

Antwort
Schlechter Testcode wird teuer, erschwert Änderungen und kann dazu führen, dass Tests deaktiviert oder nicht mehr ausgeführt werden.

CI/CD-Grundlagen

54 · Frage
Warum ist Testautomatisierung wichtig?

Antwort
Sie liefert schnelles Feedback, spart manuelle Arbeit, prüft reproduzierbar in einer definierten Umgebung und verhindert das Problem »It works on my machine«.

55 · Frage
Was bedeutet Continuous Integration?

Antwort
Änderungen werden häufig in eine gemeinsame Codebasis integriert, automatisch kompiliert und durch automatisierte Tests überprüft.

56 · Frage
Welche Voraussetzungen braucht CI?

Antwort
Ein Versionskontrollsystem, einen CI- beziehungsweise Build-Server, eine ausführbare Beschreibung des Build- und Testprozesses, bei Bedarf Build-Agenten und einen Feedback-Kanal.

57 · Frage
Was ist eine CI-Pipeline?

Antwort
Die definierte Folge von Schritten, die nach Änderungen ausgeführt wird, um Software zu bauen, zu testen und Ergebnisse zu berichten.

58 · Frage
Welche Mindeststufen hat eine CI-Pipeline?

Antwort
Mindestens Build und Test; häufig kommen Packaging, Veröffentlichung, Deployment oder zusätzliche Prüfungen hinzu.

59 · Frage
Was ist die goldene Regel bei einer fehlschlagenden CI-Pipeline?

Antwort
Sie muss sofort behoben werden, damit die gemeinsame Codebasis jederzeit vertrauenswürdig bleibt.

60 · Frage
Was bedeutet Continuous Delivery?

Antwort
Die Software wird automatisiert geprüft und in einen auslieferbaren Zustand gebracht, oft in einer produktionsnahen Testumgebung. Das Produktionsdeployment kann noch manuell ausgelöst werden.

61 · Frage
Was bedeutet Continuous Deployment?

Antwort
Nach erfolgreicher automatisierter Prüfung wird die Software automatisch in die Produktionsumgebung deployt.

62 · Frage
Warum sollte dasselbe Build-Artefakt durch mehrere Umgebungen wandern?

Antwort
Damit nicht jede Umgebung neu baut und dadurch unbemerkte Unterschiede zwischen Test und Produktion entstehen.

63 · Frage
Was ist ein Build-Artefakt?

Antwort
Das reproduzierbar erzeugte, auslieferbare Ergebnis eines Builds, zum Beispiel ein Paket, Container-Image oder Release-Archiv.

Deployment

64 · Frage
Welche Ziele hat ein gutes Deployment?

Antwort
Software soll Benutzern bereitgestellt werden, die Nutzung bestehender Benutzer möglichst wenig beeinträchtigen und bei Problemen ein Rollback auf eine funktionierende Version ermöglichen.

65 · Frage
Warum sollte Deployment automatisiert werden?

Antwort
Manuelle Deployments sind zeitaufwendig und fehleranfällig. Ein stabiles und sicheres Deployment gehört deshalb in die CI/CD-Pipeline.

66 · Frage
Was ist Reckless beziehungsweise Recreate Deployment?

Antwort
Die alte Version wird vollständig entfernt und anschließend die neue Version bereitgestellt. Die Strategie ist einfach, verursacht aber effektive Downtime und berücksichtigt die Nutzerwirkung kaum.

67 · Frage
Was ist Rolling Deployment?

Antwort

Instanzen werden schrittweise durch die neue Version ersetzt. Während des Übergangs laufen alte und neue Version parallel; dafür ist Rückwärtskompatibilität wichtig.

68 · Frage
Was ist Blue/Green Deployment?

Antwort

Zwei identische Umgebungen existieren parallel. Nach dem Test der neuen Version wird der Datenverkehr auf die neue Umgebung umgeschaltet; die alte Umgebung bleibt als Rückfalloption erhalten.

69 · Frage
Was ist Canary Deployment?

Antwort

Eine neue Version wird zuerst nur einer kleinen Benutzer- oder Instanzengruppe bereitgestellt. Nach Beobachtung wird der Anteil schrittweise erhöht.

70 · Frage
Welche Strategieeigenschaft ist bei Rolling Deployments besonders wichtig?

Antwort

Während der Übergangsphase muss das System mit alten und neuen Versionen zugleich funktionieren; APIs, Nachrichtenformate und Datenbankschemata müssen daher vorübergehend kompatibel sein.

71 · Frage
Welche Deployment-Strategien bieten eine gute Rollback-Möglichkeit?

Antwort

Vor allem Blue/Green und Canary; auch bei anderen Strategien muss ein Rückweg zu einer guten Version vorbereitet werden.